Weak Endogenous NEAT: Experiments and Results

Setup, terminology, and comparison of modes

Endogenous-Parameter NEAT prototype

August 6, 2026

Outline

1. Motivation and terminology
2. Experimental setup
3. Tasks
4. Results
5. Conclusions

Classical NEAT in one slide

NEAT (NeuroEvolution of Augmenting Topologies) evolves neural networks by mutating *topology* and *weights*.

Classical description:

$$G_t = (A_t,\, w_t), \qquad G_{t+1} = M_{\mathrm{NEAT}}(G_t;\, \lambda)$$

- A_t: network topology (nodes and connections)
- w_t: connection weights (and related gene attributes)
- λ: **fixed** mutation-control parameters set by the experimenter (in a config file)

Examples of λ: probability of adding a node, rate/power of weight mutation, ...

What is θ (theta)?

In this project, θ is a vector of *per-genome* mutation-control parameters—the endogenous analogue of λ.

Weak endogenous NEAT:

$$G_t = (A_t,\, w_t,\, \theta_t), \qquad G_{t+1} = M_{\mathrm{NEAT}}(G_t;\, \theta_t)$$

Properties of θ

- Encoded in each genome (not only in the global config)
- Inherited at crossover
- Mutated slightly between generations
- Optionally rewritten by network **meta-outputs** (`meta-theta` mode)
- Always clipped to fixed bounds

The eight components of θ

Field	Role	Typical bounds
`node_add_prob`	P(add hidden node)	[0, 0.5]
`node_delete_prob`	P(delete node)	[0, 0.5]
`conn_add_prob`	P(add connection)	[0, 0.8]
`conn_delete_prob`	P(delete connection)	[0, 0.8]
`weight_mutate_rate`	P(mutate a weight)	[0, 1]
`weight_mutate_power`	size of weight noise	[0.001, 5]
`bias_mutate_rate`	P(mutate a bias)	[0, 1]
`enable_disable_rate`	P(toggle connection)	[0, 0.5]

Defaults match the NEAT config (e.g. weight mutate rate 0.8, power 0.5).

Important claim: *weak* endogeneity

What we claim

Selected **operator-control parameters** are in the genome, inherited, mutated, selected, and (in one mode) modified by network meta-outputs.

What we do *not* claim

- Fully self-replicating neural networks
- The network implements its own reproduction algorithm
- Complete endogenous evolution

Reproduction (selection, crossover, speciation) remains an **external** NEAT implementation (`neat-python`).

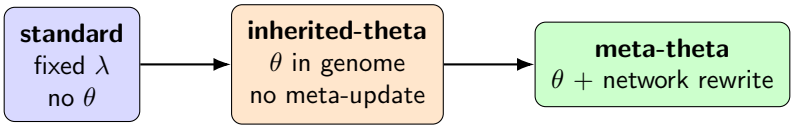
Glossary

λ	Global mutation parameters in the config (classical NEAT).
θ	Per-genome mutation parameters (this prototype).
Task output	Network output used for fitness (always `output[0]`).
Meta-outputs	Extra outputs `output[1..8]` that can set θ.
Network rewrite	In `meta-theta`: replace θ after eval from averaged meta-outputs (full replacement, not a blend).
Matched capacity	All modes use the same I/O sizes (fair comparison).
Solve	Best fitness reaches a task-specific threshold.
Generation	One evaluate $\rightarrow$ reproduce cycle.
Run	One independent evolution (own random seed / trajectory).

Software stack

- Base library: local fork of **neat-python** (submodule)
- Prototype package: `endogenous_neat`
- Extensions:
  - `Theta` dataclass
  - `EndogenousGenome` (extends `DefaultGenome`)
  - `EndogenousReproduction` (thin wrapper)
- CLI: `python -m endogenous_neat.experiment --task ... --mode ...`

Three comparison modes



- **standard**: classical NEAT; mutation rates from config only
- **inherited-theta**: θ inherited & randomly mutated; network does not write θ
- **meta-theta**: after evaluation, replace θ from meta-outputs

How θ is used at mutation time

For endogenous genomes, mutation does *not* permanently overwrite the shared config.

1. Build a local copy of the genome config
2. Overlay this genome's θ onto mutation rates
3. Call standard NEAT `mutate` with that local config
4. Apply a small random mutation to θ itself

Speciation distance also includes a small θ term (coefficient 0.1 times L_1 distance of θ vectors).

Network rewrite in `meta-theta`

Network rewrite = after evaluating a genome, its θ is *discarded and replaced* by values computed from the network's extra outputs.

Place in the generation loop:

1. Build network from genome
2. Run the task (XOR cases / cart-pole steps)
3. Fitness $\leftarrow$ $\mathtt{out}_0$ only
4. Meta-outputs $\leftarrow$ average of $\mathtt{out}_{1..8}$ over the evaluation
5. `genome.theta` $\leftarrow$ `Theta.from_meta_outputs(mean_meta)`
6. Later: reproduction uses that new θ when mutating offspring

Step 5 is the rewrite. In `inherited-theta`, step 5 is skipped.

What the network outputs mean

All modes use **9 outputs**; only `meta-theta` uses heads 1–8 for θ:

Output	Role
$\texttt{out}_0$	Task (XOR answer or pole force)
$\texttt{out}_1$	$\rightarrow$ `node_add_prob`
$\texttt{out}_2$	$\rightarrow$ `node_delete_prob`
$\texttt{out}_3$	$\rightarrow$ `conn_add_prob`
$\texttt{out}_4$	$\rightarrow$ `conn_delete_prob`
$\texttt{out}_5$	$\rightarrow$ `weight_mutate_rate`
$\texttt{out}_6$	$\rightarrow$ `weight_mutate_power`
$\texttt{out}_7$	$\rightarrow$ `bias_mutate_rate`
$\texttt{out}_8$	$\rightarrow$ `enable_disable_rate`

Averaging before the rewrite

Meta heads can change on every activation, so they are averaged **once per genome** after evaluation:

- **XOR:** mean of $\texttt{out}_{1..8}$ over the 4 input cases
- **Pole:** mean of $\texttt{out}_{1..8}$ over all timesteps of all episode runs

Result: one length-8 vector `mean_meta`. This avoids updating θ after every single activation in an order-dependent way.

Mapping meta-outputs $\rightarrow \theta$

Raw network numbers are *not* used as probabilities. Each component x_i is mapped into its legal range $[m_i, M_i]$:

$$\theta_i = m_i + \sigma(x_i)\,(M_i - m_i), \qquad \sigma(x) = \frac{1}{1 + e^{-x}}$$

- $x \approx 0 \rightarrow$ mid-range of the bound
- large positive $x \rightarrow$ near maximum
- large negative $x \rightarrow$ near minimum

Then: `genome.theta = Theta.from_meta_outputs(mean_meta)`
— a **full replacement**, not a blend with the old θ (no residual / α-update in Phase 1).

What network rewrite is *not*

- It does **not** change network weights or topology at evaluation time
- It does **not** change how fitness is computed (still only $\texttt{out}_0$)
- It only changes the genome's **mutation-control vector** θ used in the **next** reproduction step

Consequence

θ inherited from parents can be overwritten every generation. If meta-heads saturate, θ jumps toward extreme bounds (e.g. high `weight_mutate_power`) even when that hurts the task.

Matched topology capacity

To compare modes fairly, **all modes** use the same network I/O layout:

- XOR: 2 inputs, **9 outputs**
- Pole: 4 inputs, **9 outputs**

Layout:

$$\underbrace{\texttt{out}_0}_{\text{task}} + \underbrace{\texttt{out}_1, \ldots, \texttt{out}_8}_{\text{meta / } \theta}$$

- Fitness always uses $\texttt{out}_0$ only
- In `standard` / `inherited-theta`, meta heads exist but are ignored
- This controls for "extra output nodes", not classical 1-output XOR

Task 1: XOR (9-output)

Classic Boolean XOR, but with 9 network outputs.

Inputs / targets

Input	Target
$(0, 0)$	0
$(0, 1)$	1
$(1, 0)$	1
$(1, 1)$	0

Fitness

$$F = 4 - \sum (\hat{y} - y)^2$$

Solved if $F \geq 3.9$

Population size 150
Fitness plateau $F \approx 3.0 \Leftrightarrow$ constant ~ 0.5 outputs

Task 2: Single-pole balancing

Cart–pole control (feedforward), adapted from neat-python examples.

- **State** (4 inputs, scaled): cart position, velocity, pole angle, angular velocity
- **Action**: discrete force from $\texttt{out}_0$ ($+10$ or -10)
- **Fitness**: worst balance time (seconds) over 5 random starts, cap $60\,\mathrm{s}$
- **Solved** if $F \geq 60.0$
- Population size 250

Again: 9 outputs total (1 control + 8 meta) in all modes.

What we measure

Per run (CSV + plots):

- Best / mean fitness vs generation
- Best-genome θ vs generation (endogenous modes)
- Population mean / std of θ

Across modes (`--mode compare`):

- Solve rate (fraction of runs reaching threshold)
- Final best fitness (mean $\pm$ std, median)
- Median first-solve generation (among successful runs)
- Overlay plots of median fitness $\pm$ IQR and θ trajectories

Reading the plots carefully

Early stopping

When fitness hits the threshold, evolution stops. Median fitness curves for successful modes look "short"—that means **solved**, not crashed.

- Prefer **solve rate** and **time-to-threshold** over truncated median curves
- Median fitness stays near a plateau until $>$ half of runs escape it
- Mixing short smoke tests with long runs in one folder corrupts comparisons

XOR results: compare_200x50

Setup: **50 runs** $\times$ up to **200 generations** $\times$ 3 modes (fair, matched 9-output XOR)

Mode	Solved	Solve rate	Median first solve
standard	32/50	**64%**	gen 125
inherited-theta	19/50	38%	gen 140
meta-theta	2/50	4%	gen 164

Ranking: **standard** $\gg$ **inherited-theta** $\gg$ **meta-theta**

XOR: interpretation

- Fitness ≈ 3.0 is a common local plateau (constant mid-outputs)
- Encoding θ alone (`inherited-theta`) **hurts** vs fixed λ (-26 percentage points solve rate)
- Network-written θ (`meta-theta`) hurts much more (-60 pp vs standard)
- Meta-θ drifts toward high `weight_mutate_power` (~ 3) and high enable/disable rate—aggressive mutation on a bad attractor

Valid **negative result** for Phase-1 weak endogeneity on this XOR setup.

Pole results: pole_compare2

Setup: up to **200 generations**; note uneven n (20 / 10 / 10)

Mode	Solved	Solve rate	Median first solve
standard	20/20	**100%**	gen 22.5
inherited-theta	10/10	**100%**	gen 15
meta-theta	0/10	0%	—

Meta-theta finals: median $\approx 9.3\,\mathrm{s}$, mean $\approx 14.7\,\mathrm{s}$ (best unfinished run $\approx 57.5\,\mathrm{s}$).

Pole: interpretation

- Cart-pole is much easier for classical NEAT than 9-output XOR
- `inherited-theta` is **competitive** (100% solve; possibly faster)
- `meta-theta` still fails: same aggressive θ pattern as on XOR
- Fitness plot understates standard/inherited success (early stopping)

Cross-task summary

- Inherited θ: harmful on XOR, fine on pole (task-dependent)
- Meta θ: harmful on **both**

What θ does under meta-theta

Across tasks, best-genome meta-θ consistently shifts toward:

Parameter	Default / inherited	Meta-theta tendency
`weight_mutate_power`	~ 0.5	~ 2.5–3.5
`enable_disable_rate`	~ 0.01	~ 0.25–0.35
`node_add/delete_prob`	~ 0.2	higher (~ 0.3)
`weight_mutate_rate`	~ 0.8	often lower

θ *is evolving*—but not in a way that improves task fitness here.

Conclusions

1. Prototype implements **weak endogeneity**: per-genome θ, three modes, XOR + pole, automated comparisons.
2. On matched-capacity XOR, classical NEAT outperforms endogenous modes.
3. On pole, inherited θ matches classical success; meta-θ does not.
4. Meta-output replacement of θ appears misaligned with escaping plateaus (over-aggressive mutation).

Possible next steps (not claimed here): residual θ updates, tighter bounds on mutation power, task-only vs matched-capacity ablations, harder control/memory tasks.

How to reproduce

`Pole comparison python -m endogenous`$_{n}$*eat.experiment* $--$*taskpole*$--$*modecompare* $-$ $-$*generations*200 $-$ $-$*runs*10 $-$ $-$*output* $-$ *dirruns*/*pole*$_{c}$*ompare*/

Artifacts under `<output-dir>/comparison/`: `compare_summary.txt`, `compare_stats.csv`, `compare_fitness.png`, `compare_theta.png`.

Thank you

Questions?

Terminology reminder: this is *weak endogeneity* / self-adaptive operator control—not full endogenous or self-replicating NEAT.